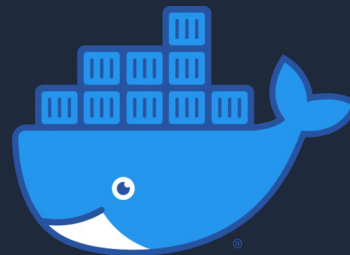

Docker i Kubernetes

Dzień 1 — Fundamenty

Maciej Krajewski · jsystems.pl



Plan dnia 1

Docker: obrazy, "co źle?", multistage, hardening

K8s podstawy: Pod, Service, Probes, Deployment

kubectl: kontekst, exec, debug

Multi-container Pod: sidecar pattern

~ 13 ćwiczeń · 7 bloków × 45 min · 09:00–16:00



Docker

Konteneryzacja w 4 godziny.

Co jest źle z tym Dockerfile?

```
FROM alpine ← (1)
RUN apk add --update nginx ← (2) (3)
RUN rm -rf /var/cache/apk/* ← (4)
RUN mkdir -p /tmp/nginx

COPY nginx.conf /etc/nginx/nginx.conf
COPY default.conf /etc/nginx/conf.d/default.conf
WORKDIR /usr/html ← (5)
COPY . ./ ← (6)
CMD nginx ← (7) kontener UMRZE!
```

Ćwiczenie D1/01: znajdź 8-10 problemów. Hint: cache, USER, EXPOSE, HEALTHCHECK, .dockerignore

Multi-stage build — Go binary do scratch

Single-stage (~900 MB)

```
FROM golang:1.22
WORKDIR /app
COPY . .
RUN go build -o app
# cały toolchain w obrazie
CMD ["/app"]
```

~900 MB

single-stage

Multi-stage (~10 MB)

```
FROM golang:1.22 AS build
WORKDIR /app
COPY . .
RUN CGO_ENABLED=0 go build -o app

FROM scratch
COPY --from=build /app/app /
CMD ["/app"]
```

~10 MB

multi-stage (90× mniej)

0

CVE w OS packages

Ćwiczenie D1/02: zbuduj oba, porównaj rozmiar i CVE (trivy)

Trivy — skan obrazów (demo na żywo)

```
$ trivy image --severity HIGH,CRITICAL --exit-code 1 vuln-app:v1
vuln-app:v1 (debian 11.5)
Total: 347 (HIGH: 198, CRITICAL: 149)
CVE-2021-44228 log4j RCE via JNDI lookup
CVE-2022-22965 spring RCE Spring4Shell
$ echo $? → 1 # CI pipeline blocked
```

Co skanuje: OS packages · language deps (npm, pip, go) · misconfig · secrets

W CI: z → blokuje merge

SARIF output → GitHub code scanning, GitLab security dashboard

 **Demo: zapuścimy skan na obrazie z log4j 2.14 — zobaczymy log4shell w akcji**

Supply chain — Cosign + SBOM

Cosign (Sigstore) — kryptograficzne podpisywanie obrazów

Keyless signing (OIDC) — krótko żyjące cert od GitHub Actions / Google

SBOM = Software Bill of Materials — lista WSZYSTKICH komponentów (CycloneDX)

Rekor — publiczny transparency log (jak CT dla TLS)

```
$ cosign sign --key cosign.key myapp:v1
$ trivy image -f cyclonedx -o sbom.json myapp:v1
$ cosign attest --predicate sbom.json --type cyclonedx myapp:v1
$ cosign verify --key cosign.pub myapp:v1 → ✓ valid
```

 Wg agendy: "podpisywanie obrazów (Cosign), SBOM (przegląd)" — sekcja 9, w D4 dyskusja Supply Chain

Hardening obrazu — checklist

Build-time

- ✓ Distroless / Alpine base
- ✓ Pinowane wersje (nginx:1.27.0)
- ✓ Multi-stage build
- ✓ .dockerignore
- ✓ USER nonroot
- ✓ Hadolint (CI lint)

Runtime / K8s

- ✓ readOnlyRootFilesystem
- ✓ capabilities: drop ALL
- ✓ seccomp: RuntimeDefault
- ✓ HEALTHCHECK
- ✓ Trivy scan (D1)
- ✓ Cosign + admission verify (D4)

Defense in depth: obraz musi być bezpieczny *nawet jeśli* uruchomi się bez SecurityContext klastra → szczegóły w D4/05



Kubernetes

Orkiestracja w 4 godziny.

Architektura Kubernetes

CONTROL PLANE

kube-apiserver · etcd
scheduler · controller-manager



Worker Node

kubelet
kube-proxy
containerd

Worker Node

kubelet
kube-proxy
containerd

Worker Node

kubelet
kube-proxy
containerd

Pod — najmniejsza jednostka

1+ kontenerów dzielących sieć (localhost) i wolumeny

Efemeryczny — restart = nowy IP (Service stabilizuje, D1/07)

Najmniejsza jednostka **schedulingu** (nie kontener!)

- Multi-container: sidecar, ambassador, adapter, init (D1/12)

```
$ kubectl apply -f pod.yaml
$ kubectl get pods
$ kubectl describe pod myapp # events, status
$ kubectl logs -f myapp # follow logs
$ kubectl exec -ti myapp -- sh # wejdź do kontenera
$ kubectl port-forward myapp 8080:80
```

Service + DNS w klastrze

DNS w klastrze: — CoreDNS automatycznie. Pod widzi search domain → wystarczy

Healthchecks — Liveness, Readiness, Startup

⚠ **Antywzorzec:** Liveness == Readiness na tym samym endpointzie → boot loop. Liveness powinien sprawdzać *żywołność procesu*, Readiness *gotowość do ruchu*.

Deployment + rolling update

```
spec:
  strategy:
    type: RollingUpdate
  rollingUpdate:
    maxSurge: 25% # dodatkowe Pody w trakcie
    maxUnavailable: 0 # zawsze 100% dostępne
```

ReplicaSet pod spodem — historia rollback (default 10 wersji)

- status / history / undo / restart
- Zmiana template → nowy ReplicaSet, stary skalowany do 0

 Pełne porównanie 6 strategii (Recreate · Rolling · Blue/Green · Canary · A/B · Shadow): D3

Multi-container Pod — 4 wzorce

Native sidecar (K8s 1.29+): z = sidecar startujący PRZED main containers, żyjący do końca Poda

Podsumowanie Dnia 1

Docker: "co jest źle?" Dockerfile, multi-stage Go → scratch, Trivy demo, Cosign

K8s podstawy: architektura, Pod, Service + DNS, Probes, Deployment

Zarządzanie: kubectl context, exec, logs, port-forward

Wzorce: multi-container Pod (sidecar, ambassador, adapter, init)

Jutro (D2): Workloady (Jobs, CronJob, StatefulSet, DaemonSet) · ConfigMap + Secrets · AuthN/AuthZ + RBAC · Gateway API + cert-manager + Let's Encrypt